# LogicPuzzleX

Workflow and timeline overview: what the whole team does, in what order, from today to the December final

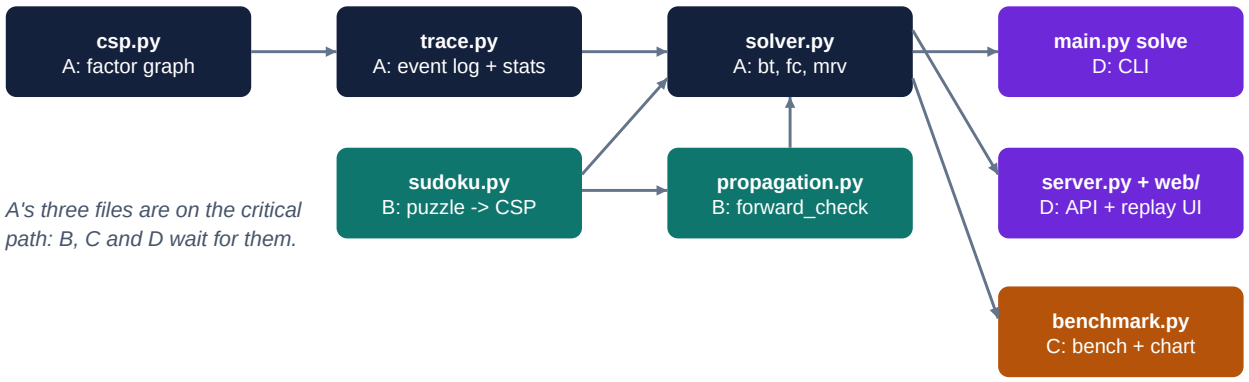
| | |
|---|---|
| Course | CSE 440 (AI), Section 1, Group 2 |
| Prepared for | All members; Member A is Shihab (csp.py, trace.py, solver.py) |
| Date | Friday 30 October 2026. Nothing has been built yet except A's csp.py and its tests. |
| Milestones | Update on Monday 2 November 2026. Final submission in December 2026 (class date to be confirmed). |
| Repository | github.com/rayedriasat/logic-puzzle-x |

## 1. Team workflow (the same for everyone)

| Step | What to do |
|---|---|
| 1 | Clone the repo. Set `git config` name and the email verified on GitHub, otherwise commits do not show on the contributor graph. |
| 2 | Add your name and ID to the README team table and commit it (your first commit). |
| 3 | One branch per task: `feat/<area>`. Never commit work directly to main. |
| 4 | Finish the task, run `python -m pytest -q`, commit with a clear message (for example `feat(solver): add MRV`), push. |
| 5 | Open a Pull Request. Another member reviews it and merges with 'Create a merge commit' or 'Rebase and merge', **never squash**. |
| 6 | Aim for at least 2 meaningful commits per member per week. The manual penalizes thin or uneven GitHub activity. |

## 2. Who depends on whom

The solver chain is the critical path. If A's three files slip, B, C and D cannot test their work, so A's tasks come first.




*A's three files are on the critical path: B, C and D wait for them.*

## 3. Roles

| Member | Owns (code) | Owns (report and slides) |
|---|---|---|
| **A: Shihab** | csp.py, trace.py, solver.py, performance | Methods, Discussion, Conclusion; slides on approach and the algorithm ladder |
| B: TBD | sudoku.py, kakuro.py, propagation.py, generator.py | Problem formulation; slides on modeling, Kakuro, generator |
| C: TBD | sat.py, explain.py, benchmark.py, data/ | Background, Experiments and results; slides on results and the logic track |
| D: TBD | server.py, web/, main.py, CI, README visuals, video | Abstract, Introduction, System design; slides on the hook, demo, takeaways |

## 4. Timeline at a glance

| | Fri 30 Oct | Sat 31 Oct | Sun 1 Nov | Mon 2 Nov | 3-16 Nov | 17-30 Nov | 1 Dec - final |
|---|---|---|---|---|---|---|---|
| Phase | Setup | Engine | Connect | UPDATE | Core (P1) | Polish; freeze | Ship |
| A: Shihab | git setup; merge csp.py + tests | trace.py; solver bt, fc, mrv | Bug fixes; verify; report Approach | Present | LCV, degree, mac/mac+, solution count | Speed, 16x16, trace cap; Methods draft | Report parts; slide 4; code tour |
| B | git setup; sudoku.py | forward_check; Sudoku tests | main.py solve works | Present | AC-3, hidden singles, Kakuro | Generator + grading | Proofread report |
| C | git setup; sample puzzles | benchmark.py; bench v1 | Chart; Preliminary results | Present | sat.py: CNF, DPLL, uniqueness | explain.py; experiments E1-E5 | Experiments section |
| D | git setup; /api/solve | UI board; replay (mock) | Real trace; screenshot | Present + GitHub tour | Race mode; Kakuro board | Explain/Generate/Bench tabs; README GIF | Video; final polish |

Colors: blue = Phase 1 MVP (update week); yellow = Monday update; teal = Phase 2 core; purple = Phase 3 polish and feature freeze; pink = Phase 4 ship. The dates after Monday are proposed. Replace them with the real class calendar, quiz and midterm weeks, and the final class date once confirmed.

## 5. What each phase must deliver

| Phase | Done when |
|---|---|
| 1. MVP (to Mon 2 Nov) | Sudoku solved from the CLI by bt, fc and mrv with correct stats; trace JSON produced; basic web board with replay; tests green in CI; one chart comparing the three algorithms; 2-page report and about 6 slides on GitHub. |
| 2. Core (3-16 Nov) | AC-3 and MAC, LCV, degree, mac+; Kakuro solved; CNF + DPLL with uniqueness proof; Race mode and Kakuro board in the UI. |
| 3. Polish (17-30 Nov) | Generator with graded unique puzzles; Explain mode; experiments E1-E5 with figures; 16x16 works; README GIF and CI badge. **Feature freeze at the end of this phase.** |
| 4. Ship (1 Dec - final) | 8-page IEEE report, about 9 slides, 1-minute video, three stopwatch rehearsals, printed report, tag v1.0-final. |

## 6. Monday update: minimum checklist

- Every member has commits and appears in the README team table; CI badge is green.
- `python main.py solve "<puzzle>" --algo mrv` solves a Sudoku in the terminal and prints nodes, backtracks and time.
- **others/update_report.pdf**: 2 pages, IEEE double-column, no title page, course header above the top margin (course and section, group number, names and IDs), all references cited including AI tools.
- **others/update_presentation.pptx**: about 6 slides, footer with group number and all names on every slide (Slide Master), start by introducing every member by full name.
- Both files uploaded to GitHub at least 12 hours before class. Present only from GitHub; no laptops.
- If it is the formal update: printed report, both-sided, no title page, no spiral binding, handed in at the very start of class.
- Slot: 5 minutes, 1 for preparation and 4 for the presentation, split equally, everyone speaks. Start on time even if the previous group overruns.

### If something is not ready by Sunday night

> The minimum that still works: commits from every member, the terminal solve with stats, green tests, and one stats table or chart for bt, fc and mrv. The web replay can slip to Phase 2.

## 7. December final: checklist

- **others/** holds five files: final_presentation.pptx, final_report.pdf, update_presentation.pptx, update_report.pdf, demo_video.mp4 (under 50 MB).
- README has a pitch, GIF, quick start, repo structure, code tour table, team and references. requirements.txt lists every library and main.py runs the project.
- Final report: 8 pages in IEEE format, no title page, course header, contributions per member, every reference cited including AI tools.
- Slot of 15 minutes: 2 prep, 6 slides, 1 video, 4 code from GitHub (README, commit history, every part of the code and repo), 2 Q&A. Start on time.
- Printed final report handed in at the very start of class; everyone attends both final classes and asks at least one question for the bonus.

## 8. Member A (Shihab): next actions in order

| # | Action | Output |
|---|---|---|
| 1 | Set git identity; add your row to the README team table | First commit visible on the contributor graph |
| 2 | Create branch feat/csp-core; add csp.py and tests/test_csp.py; run pytest; push; open a PR; ask B to review | csp.py merged to main |
| 3 | Post the interface contracts (Tracer, solve(), forward_check(), sudoku_to_csp()) in the group chat | B, C and D can code in parallel |
| 4 | Write trace.py: events assign, prune, backtrack, solved; stats counters; 100k event cap | Unit-tested Tracer |
| 5 | Write solver.py with bt, fc, mrv; add tests on the Australia map and 4x4/9x9 Sudoku | All three algorithms agree on every solution |
| 6 | Verify on the famous puzzles; fix bugs; write the Approach part of the report and slides 2-3 | Ready for the Monday update |

Prepared with the help of Claude by Anthropic; cite AI tools in the reports as the course manual requires. Full details (experiments, report page budget, video shot list, code tour, risks) are in LogicPuzzleX_Complete_Plan.pdf.